



CPU SIM

1- Introduction

CPU Sim is an interactive computer simulation package where you specify the details of the CPU to be simulated, including the register set, memory, the microinstruction set, machine instruction set, and assembly language instructions.

CPU Sim executes a program through repeated execution of machine cycles. One cycle has two parts: the fetch sequence, which is a sequence of microinstructions that typically loads the next instruction to be executed into an instruction register and decodes it, and the execute sequence, which is the sequence of microinstructions associated with the newly-decoded instruction. Execution of machine cycles halts when a halt bit is set to 1.

In this lab, you will identify the specifications of the hypothetical machine you want to simulate on CPU Sim. Using these specifications, you will create the machine hardware components, microinstructions, and instruction set. You will write a program for the hypothetical machine and the assembly language you created, run the program, and debug through it.

2- Objectives:

- 1) To enumerate different specifications of a machine.
- 2) To create a new machine on CPU Sim with a given set of specifications in a correct way.
- 3) To create an instruction set on a simulated machine on CPU Sim in a correct way.
- 4) To write an assembly language program using a given instruction set on a given machine on CPU Sim in a correct way.
- 5) To debug through a program on CPU Sim correctly.

3- Requirements:

1. Computer with windows O.S
2. CPU Sim
3. JDK 1.6 or higher version



Install CPU Sim:

- 1- Download the CPUSim onto your hard disk from the CPU Sim web page
<http://mac.softpedia.com/get/Math-Scientific/CPU-Sim.shtml>
- 2- Install Java 1.5 or later on your machine if it is not already there. Go to the web site
<http://java.sun.com/> to get the software and to read installation instructions
- 3- Fix, if necessary, your PATH system variable so that it includes the directory (installed as part of the Java installation) in which the java interpreter application is located

4- Procedure

You will use CPU Sim to run a program on a hypothetical machine. You will do the following:

- (a) create a hypothetical machine, (name it group –where n indicate your group number (1-4) -), and save it to a file
- (b) Write an assembly language program for your machine,
- (c) Assemble the program
- (d) Load it into the machine memory, and run it

4.1 Startup CPU Sim:

Double-click on the "CPUSim.bat".

The window that will appear is the main "desktop" window for CPU Sim.

4.2 Create a New Machine:

You will construct a hypothetical machine, name it group (where n integer no. from 1-4), with the basic structure shown in Figure 1 (The arrows in the figure indicate the movement of data by the microinstructions).

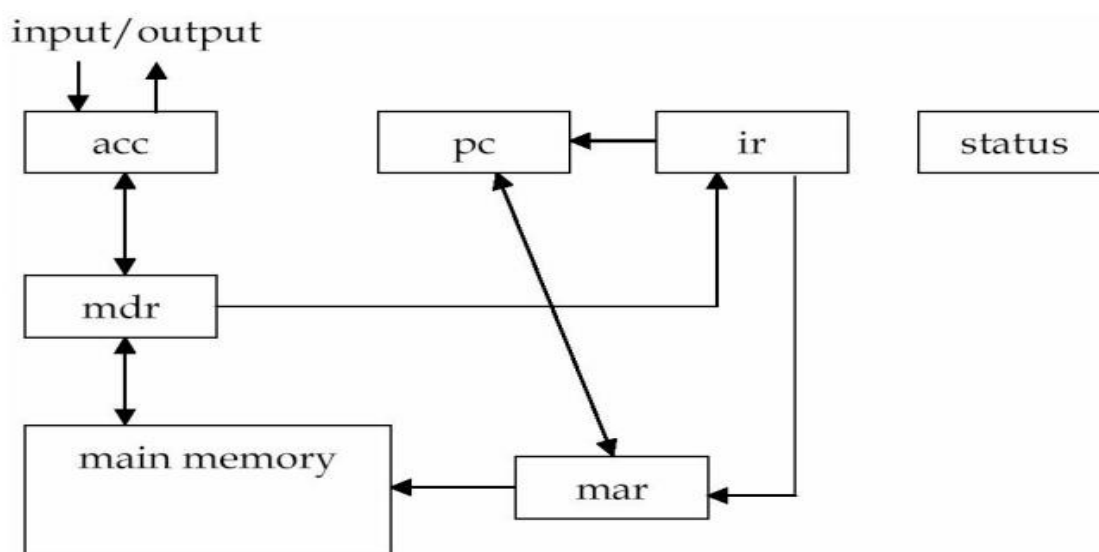


Figure 1: Diagram of data movement in the Wombat1.



4.2.1 Identify specifications

1. Registers: Six registers as following:
acc (accumulator), 16 bits.
pc (program counter), 12 bits.
ir (instruction register), 16 bits.
mar (memory address register), 12 bits
mdr (memory data register), 16 bits.
status (status register), 3 bits.
2. Main memory: 128 bytes.
3. Instruction set: 7 machine instructions, each associated with a 4-bit opcode.

Mnemonic	Description
HALT	Stop
READ	Get input from the user.
WRITE	Send output to the user
ADD	Add a value from the main memory to the value in the acc register, putting the result in the acc register.
SUB	Subtract a value from the main memory to the value in the acc register, putting the result in the acc register.
JMPZ	If the value in the acc register is 0, jump to a new location to obtain the next instruction to be executed
JUMP	Jump to a new location unconditionally

Table 1: Instruction set of the hypothetical machine.

4.2.2 Create hardware modules

1. Choose "New machine" from the File menu. The default new machine has no registers, no memory, and no machine instructions.
2. Choose "Hardware Modules..." from the Modify menu. The dialog box that appears allows you to edit the registers, register arrays, condition bits, and memory of a machine.
3. Select "Register" as the type of module and then click the "New" button six times. Edit the name and width of each of the six registers so that they match the specifications of the machine given in the previous section (see Figure 2).
4. Select "RAM" from the popup menu at the top of the Edit Modules dialog. Click "New" and set the name to "Main" and length to 128 for the new memory component .
5. Select "ConditionBit" from the popup menu at the top of the dialog. Click "New" and then set the name of the condition bit to "halt-bit", the register to the status register, the bit to 0 (it means that it is the first bit in the status register), and check the "halt" box.
6. Close the Edit Modules dialog box by clicking the "OK" button.

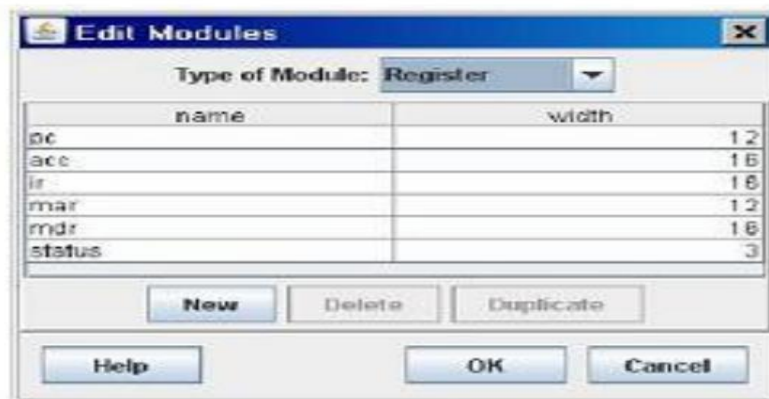


Figure 2: The dialog for editing the hardware modules.

4.2.3 Create microinstructions

Now we need to construct the necessary microinstructions that will be used to implement the machine instructions. There are:

- 7 transfer (Register to Register) The 7 transfer microinstructions are displayed by arrows between the registers in Figure 1 and show how to implement in figure 3.

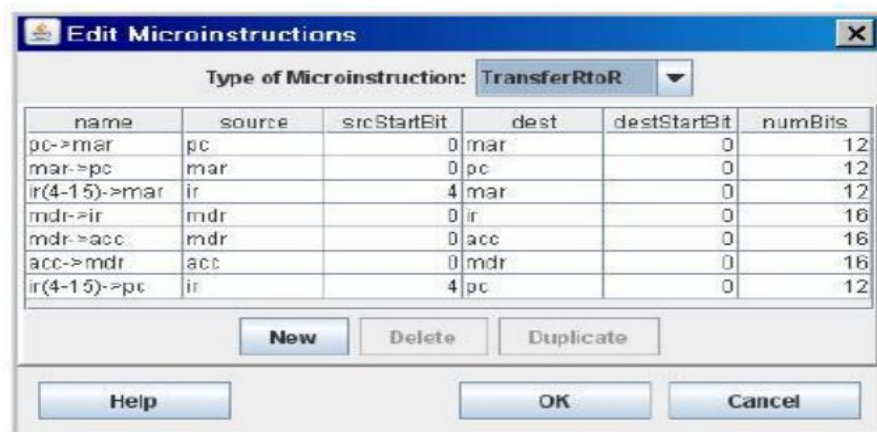


Figure 3: The dialog for editing microinstructions.

- 2 arithmetic (addition & subtraction)

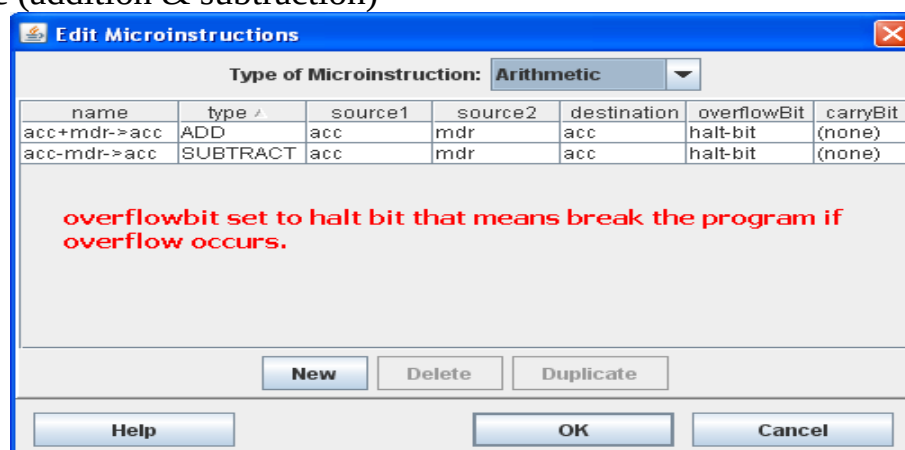


Figure 4 Arithmetic microinstruction



- 2 test (to jump microinstruction)

Edit Microinstructions

Type of Microinstruction: **Test**

name	register	start	numBits	comparison	value	omission
if(acc!=0)s...	acc	0	16	NE	0	1

Value indicate the comparable value (reference) and omission must sets to 1 where it can take three values.
0 (no jump occurs)
-1 (loop in the same micro-instruction that means executing infintly)
1 (jump if comparison result doesn't equal zero)

New Delete Duplicate

Help OK Cancel

Figure 5: Test microinstruction

- 1 increment (to increment program counter set delta to 2 that indicate increment by two where the machine is byte addressable)
- 1 decode (to decode ir)
- 2 io (Input & Output)
- 2 memory access (R/W)
- 1 set condition-bit microinstructions.

Edit Microinstructions

Type of Microinstruction: **IO**

name	type	buffer	direction
input-int->acc	integer	acc	input
output-acc->int	integer	acc	output

Type of Microinstruction: **MemoryAccess**

name	direction	memory	data	address
Main[mar]->mdr	read	Main	mdr	mar
mdr->Main[mar]	write	Main	mdr	mar

Edit Microinstructions

Type of Microinstruction: **Decode**

name	ir
decode-ir	ir

Type of Microinstruction: **Increment**

name	register	overflowBit	delta
Inc2-pc	pc	halt-bit	2

Type of Microinstruction: **SetCondBit**

name	bit	value
set-halt-bit	halt-bit	1

Figure 6: reminder microinstructions (io, memoryaccess, decode, increment and setcondbit



Create the transfer microinstructions :

You will create "TransferRtoR" type microinstructions. They are for transferring a contiguous set of bits from one register to another register

1. Choose the menu item "Microinstructions" from the Modify menu.
2. Choose "TransferRtoR" in the popup menu at the top of the dialog that appears (see Figure3).
3. Click the "New" button.
4. Edit this microinstruction so that it transfers the contents of the pc to the mar. Click in the table cell with the "?" in it (in the column headed "name") and type in "pc->mar" (you can give the microinstructions any name you want, but it helps to choose something descriptive). Select the "pc" as the "source" register and the "mar" as the "destination" register, with bit 0 as the start bit for both registers and give numBits the value 12. This means that we want to transfer all 12 bits between the registers.
5. Create the remaining 6 transfer microinstructions in your machine by adding new microinstructions in the same manner (see Figure 3)
6. The microinstructions of the other types needed for your machine can be created similarly by first selecting the appropriate type of microinstruction in the popup menu at the top of the dialog and then creating the microinstructions of that type.(see Figures from 3-6).
7. When you are finished, close the Edit Microinstructions dialog box by clicking "OK".

Question 1:

If we want to make something other than break the instruction when overflow occur what will must change (or add) in hardware modules.

(advise: we will need to set another bit in status register).

4.2.4 Construct the fetch sequence

1. Choose "Fetch Sequence" from the Modify menu .
2. The current fetch sequence list is displayed in the left scroll box entitled "Implementation". To add microinstructions to it, first display the microinstructions of the type you want in the tree of microinstructions in the right side of the dialog by clicking on the dial to the right of the folder corresponding to the desired type. Then highlight the microinstruction you want by clicking on it
3. To specify where the new microinstruction is to be inserted into the fetch sequence, click on the microinstruction in the left scroll box before which you want to insert the new microinstruction and click "<<insert<<" (see Figure 7)



4. To remove a microinstruction from the fetch sequence, click on it in the left scroll box to highlight it, and then click ">>delete". You can reorder the microinstructions in the list on the left by dragging them up or down to a new position
5. When you are done, click "OK "

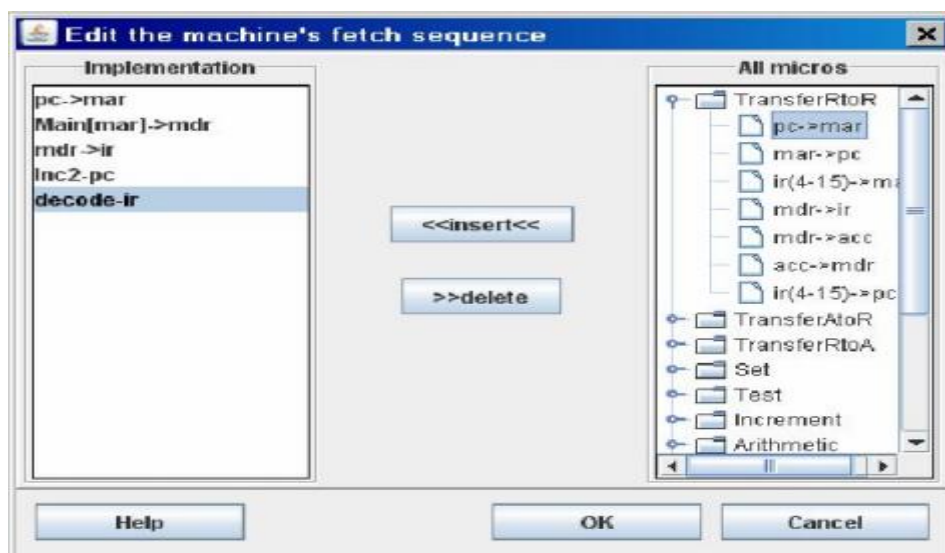


Figure 7: Fetch sequence

4.2.5 Create the instruction set

The name you specify for a machine instruction is used in assembly language programs to execute that instruction. **The opcode is specified in hexadecimal notation. The field lengths give the number of bits in each field of the instruction, including the opcode field (the first field) and all operand fields.** For example, if an instruction has field lengths of 4 and 12, then the opcode occupies the leftmost 4 bits and the remaining 12 bits of the instruction form the only operand. All machine instructions have 4-bit and 12-bit fields, but for some of the instructions, the 12-bit field is ignored (so the format of the instruction must be defined as shown in figure 8).

Edit machine fields					
name	type	numBits	relativity	defaultValue	signed
addr	required	12	absolute	0	☐
unused	ignored	12	absolute	0	☒
op	required	4	absolute	0	☐

Figure 8: format field defination

1. Choose "Machine Instructions..." from the Modify menu. The dialog box that appears allows you to edit machine instructions, including the name, opcode, field lengths, and the list of microinstructions that form the execution sequence of each instruction



- To create a new machine instruction, click "New" and then edit the parameters of that instruction. The "Opcode" is a non-negative integer (in hexadecimal) that fit in the first field of the instruction. The "Field Lengths" is a list of 1 or more positive integers whose **sum is a multiple of 8** (byte addressable machine – it is required to be 16 bit length as ir register).
- Once all machine instructions have been created, click "OK".

Example: (if we use fields as in figure 8 –you can rename these fields as you like)

- For add instruction format will be **op add** that means 4 bits for opcode and 12 bits for operand address where the summation equal 16 bits.
- For Read instruction format will be **op unused** that means 4 bits for opcode and 12 bits without using where the summation equal 16 bits multiple of 8.

Question 2

For every instruction you create, record in a table the values you use for its parameters

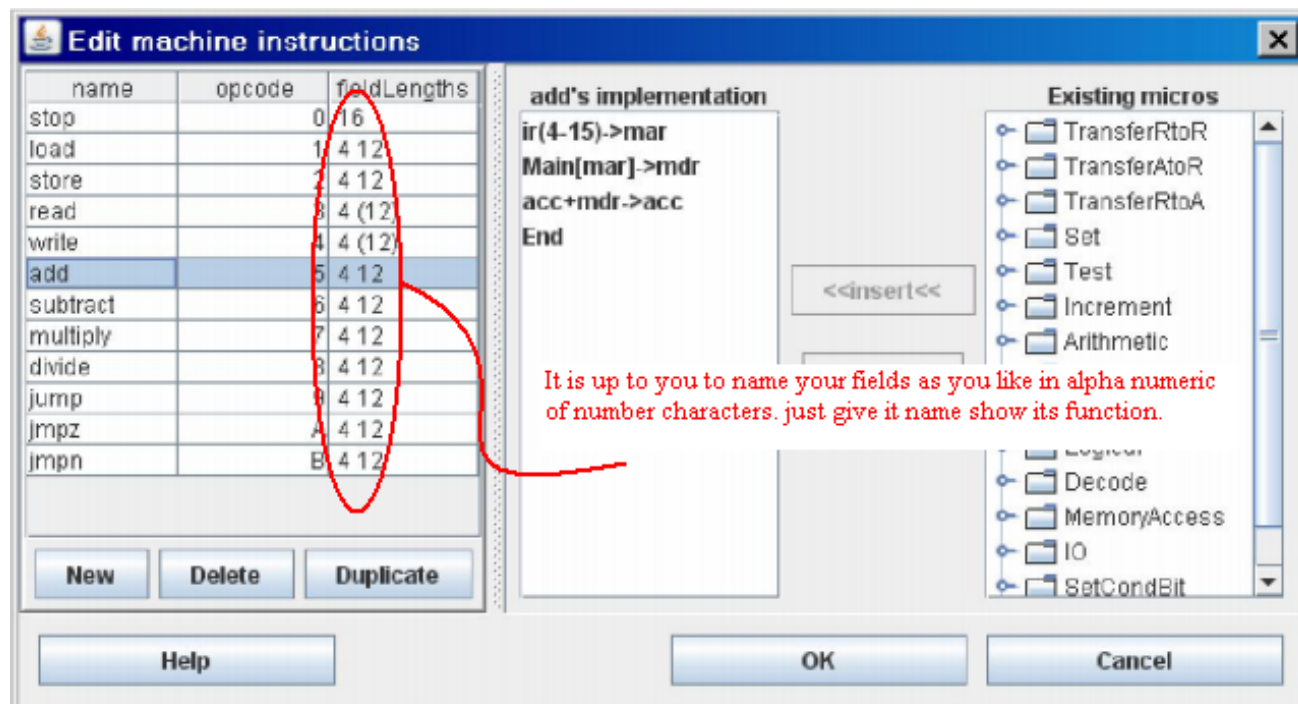


Figure 9: example of instruction set



4.2.6 Save your machine to a file

1. Choose "Save machine" or "Save machine as" from the File menu. Use the file name: Groupn.cpu (*where n integer no. from 1-4 indicate your group*).
2. CPU Sim saves the details of the machine as text in an XML file
3. Close CPU Sim

To see all the details of the machine in a more user-friendly format than XML, choose "Save machine in HTML" from the File menu. You will be asked to type in the name of the HTML file into which the data will be saved. This file can be opened and viewed with any web browser

4.3 Run a Program

1. Startup CPU Sim.
2. Choose "Open machine" from the File menu.
3. Select and open the text file you named for the Groupn machine. If the machine loads without error, then the title of the main display should become "Groupn" and two windows should appear inside the desktop.
4. Choose "New text" from the File menu
5. In the text window, write the following program:

```
; This program reads an integer and it with content of memory location  
; Then it outputs  
; the sum (not including the last number).  
start:      read          ; read n -> acc  
            add   sum      ; add sum to the acc  
            store sum      ; store the new sum  
            write         ; write the final sum  
            stop          ; end the program  
sum:        .data 2n n     ; 2-byte location where sum is stored
```



6. Save your program by selecting "Save text" from the File menu. Name it Wn
7. Choose "Assemble & Load" from the Execute menu. You should see numbers and comments appear in the First few rows of the table in the RAM window. The numbers in the data column of the RAM window are the machine language instructions generated by the assembler from the assembly language program. The comments column of the RAM window displays the lines of assembly code from which the corresponding machine instruction was generated
8. Make sure all the registers have been cleared (set to 0). If some of them are not 0, then either edit the values to make them 0 or choose "Clear all registers & arrays" from the Execute menu
9. Choose "Run" from the Execute menu. The program will begin execution with the instruction whose address (namely, 0) is stored in the program counter pc. The machine runs by repeatedly executing machine cycles consisting of the fetch sequence, which loads into the ir the instruction whose address is in the pc and then decodes the instruction, followed by the execute sequence, which executes the machine instruction that was just decoded
10. When the console panel asks you for input, type a positive integer and press the enter key. Repeat this process several times and then type in a negative number
11. When execution is complete, watch the final state of the registers and RAM in the registers and RAM windows.

Question 3:

Record these results in a table and show the place of sum in memory from the last instruction. What is the reason for the sum place in memory?

12. To return the program with different inputs, select "Clear, assemble, load & run" from the Execute menu
13. To quit CPU Sim, choose "Quit" from the File menu

4.4 Debug a Program

1. Load the Groupn machine and the Wn program.
2. Select "Clear everything" from the Execute menu
3. Choose "Assemble & load" from the Execute menu.
4. Select "Debug Mode" from the Execute menu. You will see a toolbar appear at the top of the display (see Figure 10).



Figure 10 The debugging toolbar.

5. On its right end, the debugging toolbar says that the next instruction is the fetch sequence. Click "Step by Instr" from the toolbar, then a complete machine cycle will be executed.
6. The "read" instruction will be fetched, decoded, and executed. Type the inputs in the console and press enter.
7. Each click of "Step by Instr" causes a full machine cycle to be executed.

Question 4:

In a table, record the values of the registers after executing every instruction.

5- Task

1. Each student show his(her) work for the TA personally. (individual)

In your lab report, answer Questions 1 through 4 mentioned in the Procedure section. Submit your report to the TA within (Deadline) 2 weeks of your lab date. Submit just one lab report for your group. (accept hard-copy or soft-copy)

